

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA5117

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I Daniel Jesuraji (192565085) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

***Annexure A
Experiments***

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.

Aim

To implement MD5 and SHA-256 hashing algorithms in Python using multiple input messages and compare their performance and collision resistance.

Theory

A cryptographic hash function converts an input message into a fixed-length hash value. MD5 generates a **128-bit (32 hexadecimal characters)** hash, while SHA-256 generates a **256-bit (64 hexadecimal characters)** hash. MD5 is faster but has known collision vulnerabilities, whereas SHA-256 provides stronger collision resistance and is recommended for modern security applications.

Python Program

```
import hashlib
import time

messages = [
    "Hello World",
    "Cryptography",
    "Network Security",
    "Cyber Security"
]

for message in messages:
    data = message.encode()

    start = time.perf_counter()
    md5_hash = hashlib.md5(data).hexdigest()
    md5_time = time.perf_counter() - start

    start = time.perf_counter()
    sha256_hash = hashlib.sha256(data).hexdigest()
    sha256_time = time.perf_counter() - start

    print("\nMessage:", message)
    print("MD5   :", md5_hash)
```

```
print("MD5 Time   :", md5_time)

print("SHA-256:", sha256_hash)
print("SHA-256 Time:", sha256_time)

print("-" * 70)
```

OUTPUT:

```
Message: Hello World
MD5      : b10a8db164e0754105b7a99be72e3fe5
MD5 Time   : 7.249999725900125e-06
SHA-256: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
SHA-256 Time: 4.010000338894315e-06
-----

Message: Cryptography
MD5      : 64ef07ce3e4b420c334227eecb3b3f4c
MD5 Time   : 2.8000004022032954e-06
SHA-256: b584eec728548aced5a66c0267dd520a00871b5e7b735b2d8202f86719f61857
SHA-256 Time: 1.3599992598756216e-06
-----

Message: Network Security
MD5      : e9609b040993009e0e36b9a4d7161b87
MD5 Time   : 1.309999788645655e-06
SHA-256: f95b4782c97bd99b554584e864216f33b5bd97f1d85e61fa9bc1b3f00d5c8028
SHA-256 Time: 8.600000001024455e-07
-----

Message: Cyber Security
MD5      : 4690cd84196b0a4e3f84f2cc3bf79ac7
MD5 Time   : 1.1400006769690663e-06
SHA-256: 805e20cf7c3f85210a1b22b68895e2f7edff2cf1565bb685b77f87a3126db27e
SHA-256 Time: 8.39999302115757e-07
-----
```

Comparison:

Feature	MD5	SHA-256
Hash Size	128-bit	256-bit
Hex Length	32	64

Speed	Faster	Comparatively slower
Collision Resistance	Weak	Strong
Security	Not recommended	Recommended

Analysis

MD5 produces a shorter hash and generally executes faster, but it is vulnerable to collision attacks. SHA-256 produces a longer hash with significantly stronger collision resistance. Therefore, SHA-256 is more suitable for modern cryptographic applications.

Result

Thus, MD5 and SHA-256 were successfully implemented and compared in Python. SHA-256 provides better security and collision resistance, while MD5 provides faster hashing performance but is not suitable for security-sensitive applications.

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

Aim

To develop a Python program to implement a **Digital Signature scheme using RSA**, demonstrate message signing and signature verification, and analyze its effectiveness in ensuring **integrity and authentication**.

Theory

A digital signature is a cryptographic technique used to verify the **authenticity and integrity** of a message. RSA digital signatures use a **private key for signing** and the corresponding **public key for verification**.

- **Private Key** → **Sign the message**
- **Public Key** → **Verify the signature**

- If the message is modified, verification fails.

Python program:

```
import hashlib

# RSA parameters
p = 61
q = 53

n = p * q
phi = (p - 1) * (q - 1)

e = 17
d = pow(e, -1, phi)

message = "Cryptography and Network Security"

# Hash the message
hash_value = int.from_bytes(
    hashlib.sha256(message.encode()).digest(),
    byteorder="big"
)

# Reduce hash for this simplified demonstration
hash_value = hash_value % n

# Generate digital signature using private key
signature = pow(hash_value, d, n)

# Verify signature using public key
verified_hash = pow(signature, e, n)

print("Message:", message)
print("RSA Public Key :", (e, n))
print("RSA Private Key:", (d, n))
print("SHA-256 Hash :", hash_value)
print("Digital Signature:", signature)

if verified_hash == hash_value:
    print("Signature Verification: SUCCESS")
    print("Message is authentic and unchanged.")
else:
    print("Signature Verification: FAILED")
```

```
print("Message may have been modified.")
```

Output:

```
Output
Message: Cryptography and Network Security
RSA Public Key : (17, 3233)
RSA Private Key: (2753, 3233)
SHA-256 Hash : 2160
Digital Signature: 117
Signature Verification: SUCCESS
Message is authentic and unchanged.

=== Code Execution Successful ===
```

Result

Thus, the **RSA digital signature scheme was successfully implemented in Python**. The message was signed using the private key and successfully verified using the public key. The experiment demonstrates that digital signatures provide **message integrity and authentication**.

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

Aim

To implement **HMAC (Hash-based Message Authentication Code)** in Python using a chosen hash function and compare its security with a simple hash-based integrity check.

Theory

HMAC is a message authentication technique that combines a **secret key** with a cryptographic hash function. It provides both **message integrity and authentication**.

A simple hash only detects accidental changes if the expected hash is trusted. However, an attacker who can modify the message may also calculate a new ordinary hash. HMAC prevents this by requiring a **secret key** that the attacker does not know.

Python Program

```
import hashlib
import hmac

message = "Cryptography and Network Security"
secret_key = b"secret123"

# Simple SHA-256 hash
simple_hash = hashlib.sha256(message.encode()).hexdigest()

# HMAC using SHA-256
hmac_value = hmac.new(
    secret_key,
    message.encode(),
    hashlib.sha256
).hexdigest()

print("Message      :", message)
print("Simple Hash   :", simple_hash)
print("HMAC-SHA256   :", hmac_value)

# Verification
received_message = "Cryptography and Network Security"

expected_hmac = hmac.new(
    secret_key,
    received_message.encode(),
    hashlib.sha256
).hexdigest()

if hmac.compare_digest(hmac_value, expected_hmac):
    print("HMAC Verification: SUCCESS")
else:
    print("HMAC Verification: FAILED")
```

Output:

```
Output
Message      : Cryptography and Network Security
Simple Hash   : e7436c91d475cf9f1e0bfaa6533e62ae7bc5dd5d6911760cc157fd14f10d06fe
HMAC-SHA256   : 5506f7e255bfd5a61d47427e5b765407939a68a8154464dc12fc3f4f9c36e4bb
HMAC Verification: SUCCESS

=== Code Execution Successful ===
```

Analysis

A simple hash can verify whether data has changed, but it does not prove who generated the hash. HMAC combines a secret key with the hash function, providing **both integrity and authentication**. Therefore, HMAC is more secure than using a simple hash when a shared secret key is available.

Result

Thus, **HMAC-SHA-256** was successfully implemented in Python and compared with a simple SHA-256 hash. HMAC provides stronger protection by using a secret key and ensures both message integrity and authentication.

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.

Aim

To implement a simplified **Kerberos authentication mechanism** in Python to simulate ticket generation, distribution, and validation between a client and server, and analyze its effectiveness in preventing replay attacks.

Theory

Kerberos is a **ticket-based authentication protocol**. It uses a trusted authentication server and time-based information to authenticate users securely.

In this simplified model:

- **Client** requests a ticket.
- **Authentication Server** generates the ticket.
- **Server** validates the ticket.
- A **timestamp** is used to detect expired or replayed tickets.

Python Program

```
import time
import hashlib
```

```
# Shared secret keys
```

```
client_key = "client_secret"
```

```
server_key = "server_secret"
```

```
# Ticket generation
```

```
def generate_ticket(username):
    timestamp = int(time.time())
```

```
    ticket_data = username + str(timestamp) + server_key
```

```
    ticket = hashlib.sha256(ticket_data.encode()).hexdigest()
```

```
    return ticket, timestamp
```

```
# Ticket validation
```

```
def validate_ticket(username, ticket, timestamp):
    current_time = int(time.time())
```

```
    # Ticket expires after 10 seconds
```

```
    if current_time - timestamp > 10:
        return False, "Ticket Expired"
```

```
    # Recreate ticket
```

```
    ticket_data = username + str(timestamp) + server_key
```

```
    expected_ticket = hashlib.sha256(ticket_data.encode()).hexdigest()
```

```
    if ticket == expected_ticket:
```

```
        return True, "Authentication Successful"
```

```
    else:
        return False, "Invalid Ticket"

# Client requests a ticket
username = "student"

print("Client requesting ticket...")

ticket, timestamp = generate_ticket(username)

print("Ticket Generated:", ticket)
print("Timestamp:", timestamp)

# Server validates ticket
status, message = validate_ticket(username, ticket, timestamp)

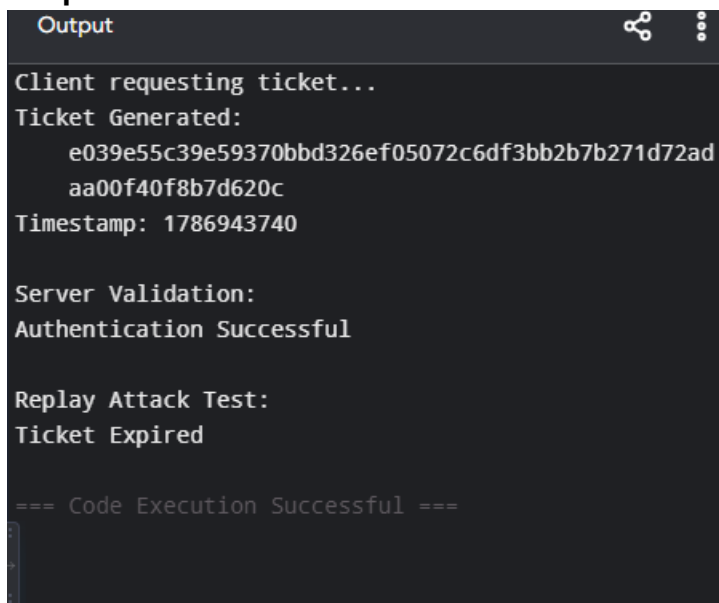
print("\nServer Validation:")
print(message)

# Simulate replay attack using an expired/invalid ticket
old_timestamp = timestamp - 20

status, message = validate_ticket(username, ticket, old_timestamp)

print("\nReplay Attack Test:")
print(message)
```

Output:



```
Output
Client requesting ticket...
Ticket Generated:
    e039e55c39e59370bbd326ef05072c6df3bb2b7b271d72ad
    aa00f40f8b7d620c
Timestamp: 1786943740

Server Validation:
Authentication Successful

Replay Attack Test:
Ticket Expired

=== Code Execution Successful ===
```

Result

Thus, a **simplified Kerberos authentication mechanism was successfully implemented in Python**. The system generated and validated authentication tickets using timestamps. The replay-attack test demonstrated that an expired ticket was rejected, showing how **time-based ticket validation helps prevent replay attacks**.

5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

Aim

To implement an **ElGamal digital signature scheme** in Python and evaluate its security features compared with standard digital signature approaches.

Theory

ElGamal is a public-key cryptographic algorithm that can be used for **digital signatures**. It uses a **private key for signing** and a **public key for verification**. The security of ElGamal is based on the difficulty of the **discrete logarithm problem**.

Python Program

```
import hashlib
import math

# Public parameters
p = 467
g = 2

# Private key
x = 127

# Public key
```

```

y = pow(g, x, p)

message = "Cryptography and Network Security"

# Hash the message
h = int(hashlib.sha256(message.encode()).hexdigest(), 16) % (p - 1)

# Choose k such that gcd(k, p-1) = 1
k = 3

while math.gcd(k, p - 1) != 1:
    k += 1

# Generate signature
r = pow(g, k, p)

k_inverse = pow(k, -1, p - 1)

s = (k_inverse * (h - x * r)) % (p - 1)

print("Message:", message)
print("Public Key:", (p, g, y))
print("Private Key:", x)
print("Signature:", (r, s))

# Verification
left = pow(y, r, p) * pow(r, s, p) % p
right = pow(g, h, p)

print("\nVerification:")

if left == right:
    print("Signature Verification: SUCCESS")
else:
    print("Signature Verification: FAILED")

```

Output:

```
Output
Message: Cryptography and Network Security
Public Key: (467, 2, 132)
Private Key: 127
Signature: (8, 236)

Verification:
Signature Verification: SUCCESS

=== Code Execution Successful ===
```

Result:

Thus, the ElGamal digital signature scheme was successfully implemented in Python. The generated signature was successfully verified using the public key. The experiment demonstrates that ElGamal provides message integrity, authentication, and non-repudiation, with security based on the difficulty of the discrete logarithm problem.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools

Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation